A Mid-Semester Report On

Lattice Based Post-Quantum Cryptography

By

Shashwat Agrawal – 2024ADPS0197H
Meghraj Goswami – 2022A2B41869H
Add your names – 202XYYZZ0000H

Under the supervision of

Prof. Sabyasachi Dey

Submitted in Partial Fulfillment of the Requirements of

MATH F366: LABORATORY PROJECT

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI (RAJASTHAN)

HYDERABAD CAMPUS

(March 2026)

Contents

I Applications of LLL to Cryptanalysis – Shashwat Agrawal 2

1 Overview 2

2 Congruential Cryptosystems 2
- 2.1 Worked Example: Breaking the Congruential Cipher . . . 2

3 Knapsack (Subset-Sum) Problems 3
- 3.1 Worked Example: Solving the Knapsack with LLL . . . 3

4 GGH Cryptosystem 3
- 4.1 Worked Example: Breaking GGH with Babai's Algorithm . . . 4

5 Conclusion 4

II NTRU Cryptosystem – Meghraj Goswami 5

6 Introduction 5

7 Mathematical Prerequisites 5
- 7.1 Groups . . . 5
- 7.2 Rings and Fields . . . 6
- 7.3 Polynomial Rings . . . 6
- 7.4 Convolution Polynomial Rings . . . 6
- 7.5 Ternary Polynomials . . . 6
- 7.6 Polynomial Inversion . . . 7

8 Cryptosystem 7
- 8.1 Public Parameters . . . 7
- 8.2 Key Generation . . . 7
- 8.3 Encryption . . . 7
- 8.4 Decryption . . . 8
- 8.5 Correctness . . . 8

9 Worked Example 8
- 9.1 Key Generation . . . 9
- 9.2 Encryption . . . 9
- 9.3 Decryption . . . 9

10 Security Analysis 9
- 10.1 Key Recovery Problem . . . 9
- 10.2 Brute Force and Meet-in-the-Middle . . . 10

11 Conclusion 10

Part I
Applications of LLL to Cryptanalysis – Shashwat Agrawal

1 Overview

The Lenstra-Lenstra-Lovász (LLL) lattice basis reduction algorithm is a foundational tool in cryptanalysis. It has successfully compromised older knapsack-based public key cryptosystems and remains the primary analytical tool against modern lattice-based schemes, including Ajtai-Dwork, GGH, and NTRU. Furthermore, LLL can be leveraged to attack RSA under specific conditions.

It is important to note that the algorithmic examples outlined in this section use very low-dimensional lattices, allowing LLL to break them trivially. In practical deployments, ensuring security requires lattice dimensions between 500 and 1000, which (with the exception of NTRU) often results in impractically large key lengths.

2 Congruential Cryptosystems

In a standard congruential cipher, Alice selects a public modulus q alongside two small secret integers, f and g. The public key is broadcast as $h \equiv f^{-1}g \pmod{q}$. Eve's objective is to compute the private key f knowing only q and h.

This can be mapped to a lattice problem. Eve constructs a lattice L spanned by the vectors:

$$\mathbf{v}_1 = (1, h) \quad \text{and} \quad \mathbf{v}_2 = (0, q)$$

Because the vector (f, g) is a linear combination of this basis and is constrained to be small, it will likely represent the shortest nonzero vector in L.

2.1 Worked Example: Breaking the Congruential Cipher

Problem Setup: Let $q = 122430513841$ and $h = 39245579300$. How can an attacker recover the private keys f and g?

Solution via LLL: Eve generates the lattice from the vectors:

$$\mathbf{v}_1 = (1,\ 39245579300) \quad \text{and} \quad \mathbf{v}_2 = (0,\ 122430513841)$$

Applying Gaussian lattice reduction (a 2D analog of LLL) takes exactly 11 iterations. The algorithm outputs a new, reduced basis containing much shorter vectors:

$$(-231231,\ -195698) \quad \text{and} \quad (-368222,\ 217835)$$

The shortest vector in this reduced basis represents the secret variables. Ignoring the irrelevant change of sign, Eve successfully retrieves Alice's private keys:

$$f = 231231 \quad \text{and} \quad g = 195698$$

3 Knapsack (Subset-Sum) Problems

The knapsack problem defined by a set of weights $M = (m_1, \ldots, m_n)$ and a target sum S can similarly be framed as a lattice reduction task. By assembling a specific basis matrix, the target solution vector is embedded into the lattice $L_{M,S}$. Because this target vector has a length of $\sqrt{n}$, it is statistically likely to be about half the size of any other nonzero vector in the lattice, making it an ideal target for LLL.

3.1 Worked Example: Solving the Knapsack with LLL

Problem Setup: Consider the knapsack instance with weight vector $M = (89, 243, 212, 150, 245)$ and target sum $S = 546$. Find the specific combination of weights that sums exactly to S.

Solution via LLL: We construct a basis matrix $A_{M,S}$ where the weights and target are aligned to form the lattice constraints:

$$A_{M,S} = \begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 89 \\ 0 & 2 & 0 & 0 & 0 & 243 \\ 0 & 0 & 2 & 0 & 0 & 212 \\ 0 & 0 & 0 & 2 & 0 & 150 \\ 0 & 0 & 0 & 0 & 2 & 245 \\ 1 & 1 & 1 & 1 & 1 & 546 \end{pmatrix}$$

Applying the LLL algorithm to the rows of this matrix takes 21 swaps and outputs the following reduced basis matrix:

$$\begin{pmatrix} -1 & 1 & -1 & 1 & -1 & 0 \\ 1 & -1 & -1 & 1 & -1 & -1 \\ -1 & -1 & -1 & 1 & 1 & 2 \\ 1 & -1 & -1 & -1 & -1 & 2 \\ -2 & -2 & 4 & 0 & -2 & 0 \\ -6 & -4 & -6 & -6 & 0 & -3 \end{pmatrix}$$

The shortest vector in the reduced basis (the top row) is $\mathbf{v} = (-1, 1, -1, 1, -1, 0)$.

To find the subset-sum solution, we write $\mathbf{v}$ as a linear combination of the original basis vectors (the rows of $A_{M,S}$). This linear combination vector is $(-1, 0, -1, 0, -1, 1)$. This translates directly to the correct subset sum:

$$-89 - 212 - 245 + 546 = 0$$

Thus, the items weighing 89, 212, and 245 are the ones that perfectly fill the knapsack.

Remark: When using LLL to solve subset-sum problems, it is helpful to multiply $m_1, \ldots, m_n$ and the target S by a large constant C. This scales up the expected length of non-target shortest vectors by $C^{1/(n+1)}$, while the target vector's length remains $\sqrt{n}$. The larger distance between the target vector and the rest of the lattice makes it significantly easier for LLL to isolate the correct solution.

4 GGH Cryptosystem

The Goldreich-Goldwasser-Halevi (GGH) cryptosystem relies heavily on the difficulty of the Closest Vector Problem (CVP). Alice publishes a 'bad' (highly non-orthogonal) basis for a lattice, and Bob encrypts a message into a vector $\mathbf{e}$ that is close to a lattice point.

4.1 Worked Example: Breaking GGH with Babai's Algorithm

Problem Setup: Alice's public lattice L is generated by the rows of the following matrix (representing a bad basis):

$$\begin{pmatrix} -4179163 & -1882253 & 583183 \\ -3184353 & -1434201 & 444361 \\ -5277320 & -2376852 & 736426 \end{pmatrix}$$

Bob's encrypted message is the vector $\mathbf{e} = (-79081427,\ -35617462,\ 11035473)$. How can Eve find the plaintext message?

Solution via LLL: Eve cannot effectively use Babai's algorithm on the bad public basis. First, she applies LLL to the lattice L to find a quasi-orthogonal 'good' basis:

$$\begin{pmatrix} 36 & -30 & -86 \\ 61 & 11 & 67 \\ -10 & 102 & -40 \end{pmatrix}$$

This new basis has a Hadamard ratio of $H = 0.956083$, meaning it is highly orthogonal. Next, Eve applies Babai's algorithm to find a lattice vector $\mathbf{v}$ that is extremely close to the intercepted ciphertext $\mathbf{e}$. Babai's algorithm returns:

$$\mathbf{v} = (79081423,\ 35617459,\ -11035471)$$

Finally, she writes $\mathbf{v}$ in terms of the original good basis vectors, which immediately retrieves Bob's plaintext:

$$\mathbf{m} = (-86,\ 35,\ 32)$$

5 Conclusion

The LLL algorithm has profoundly shaped the landscape of modern cryptography. As demonstrated by its attacks on Congruential, Knapsack, and GGH systems, translating cryptographic parameters into lattice basis vectors allows cryptanalysts to exploit the mathematical predictability of short vectors. While increasing lattice dimensions to 500 or 1000 can safeguard systems against LLL, doing so often introduces severe inefficiencies in key size. This reality underscores why cryptosystems must be evaluated not just for theoretical soundness, but for resilience against practical lattice basis reduction.

Part II
NTRU Cryptosystem – Meghraj Goswami

6 Introduction

The rapid advancement of quantum computing poses a threat to modern cryptographic infrastructure. Shor's algorithm demonstrates that a quantum computer can factor large integers and compute discrete logarithms in polynomial time—$O((\log n)^3)$—completely breaking RSA and Elliptic Curve Cryptography (ECC) at any practical key size in a short time frame [1].

In response, the cryptographic community has shifted towards **Post-Quantum Cryptography (PQC)**, algorithms that remain secure even against quantum adversaries. The **NTRU** cryptosystem (N-th degree Truncated polynomial Ring Unit), introduced by Hoffstein, Pipher, and Silverman in 1996 [2], is one of the oldest and most analysed PQC proposals. Its security rests on the hardness of the Shortest Vector Problem (SVP) in certain lattices, for which no efficient quantum algorithm is known as of yet.

Key advantages of NTRU over RSA and ECC

- No known quantum speedup for the underlying lattice problem.
- Fast polynomial operations.
- Compact keys: public keys are approximately 700 bytes at the 128-bit security level, versus several kilobytes for RSA at equivalent security.
- NTRU has been a prominent fixture in the NIST PQC standardization process.[3]

7 Mathematical Prerequisites

7.1 Groups

Definition 7.1 (Group)**.** A **group** is a pair $(G, \cdot)$ where G is a set and $\cdot : G \times G \to G$ is a binary operation satisfying:

1. **Closure:** $a \cdot b \in G$ for all $a, b \in G$.
2. **Associativity:** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
3. **Identity:** $\exists\, e \in G$ such that $a \cdot e = e \cdot a = a$.
4. **Inverses:** For each $a \in G$, $\exists\, a^{-1} \in G$ with $a \cdot a^{-1} = e$.

If additionally $a \cdot b = b \cdot a$ for all $a, b \in G$, the group is called **abelian**.

Example 7.1 (Clock arithmetic ($\mathbb{Z}/12\mathbb{Z}$, $+$))**.** Integers modulo 12 under addition: $10 + 4 \equiv 2 \pmod{12}$.
The identity is 0; the inverse of 3 is 9 since $3 + 9 \equiv 0$. This is an abelian group.

7.2 Rings and Fields

Definition 7.2 (Ring)**.** A **ring** is a triplet $(R, +, \cdot)$ where $(R, +)$ is an abelian group, multiplication is associative, and multiplication distributes over addition. A ring is **commutative** if $ab = ba$ for all $a, b \in R$.

A **field** is a commutative ring where every nonzero element has a multiplicative inverse.

Example 7.2. $\mathbb{Z}$, $\mathbb{Z}/n\mathbb{Z}$, $\mathbb{Q}$, and $\mathbb{R}$ are rings. $\mathbb{Q}$ and $\mathbb{R}$ are fields; $\mathbb{Z}$ is not (the element 2 has no multiplicative inverse in $\mathbb{Z}$).

7.3 Polynomial Rings

Definition 7.3 (Polynomial Ring)**.** Let R be a commutative ring. The polynomial ring $R[x]$ consists of all formal sums $a_0 + a_1x + \cdots + a_nx^n$ where $a_i \in R$, under coefficient-wise addition and standard polynomial multiplication.

7.4 Convolution Polynomial Rings

NTRU operates in quotient rings of the following form. Fix a positive integer N and define:

$$\mathcal{R} = \frac{\mathbb{Z}[x]}{(x^N - 1)}, \qquad \mathcal{R}_q = \frac{(\mathbb{Z}/q\mathbb{Z})[x]}{(x^N - 1)}, \qquad \mathcal{R}_p = \frac{(\mathbb{Z}/p\mathbb{Z})[x]}{(x^N - 1)}.$$

The ideal $(x^N - 1)$ enforces $x^N = 1$, so all exponents reduce modulo N, and every element can be represented as a polynomial of degree at most $N - 1$. Meanwhile, the coefficients are taken modulo q or p in $\mathcal{R}_q$ and $\mathcal{R}_p$ respectively.

Multiplication in these rings is the **convolution product** $\star$:

$$c(x) = a(x) \star b(x), \quad c_k = \sum_{\substack{i+j \equiv k \\ (\mathrm{mod}\ N)}} a_i b_j.$$

The wrap-around of higher-degree terms is what gives this product a 'cyclic' character.

Example 7.3 (Convolution product with $N = 5$)**.** Let $a(x) = 1 - 2x + 4x^3 - x^4$ and $b(x) = 3 + 4x - 2x^2 + 5x^3 + 2x^4$. Computing coefficient by coefficient using $x^5 = 1$:

$$a(x) \star b(x) = -13 + 20x - 7x^2 + 19x^3 + 5x^4.$$

7.5 Ternary Polynomials

Definition 7.4 (Ternary polynomial set)**.** For non-negative integers d_1, d_2 with $d_1 + d_2 \leq N$,

$$T(d_1, d_2) = \big\{a(x) \in \mathcal{R} \mid \text{exactly } d_1 \text{ coefficients equal } +1,\ d_2 \text{ equal } -1,\ \text{rest } 0\big\}.$$

These sparse polynomials have small coefficients. Private keys are chosen from $T(d_1, d_2)$ precisely because small coefficients make decryption work (see Section 8.5) and make exhaustive search difficult (see Section 10).

7.6 Polynomial Inversion

Key generation requires computing $f(x)^{-1}$ modulo both q and p. A polynomial $F_q(x)$ is an inverse of $f(x)$ in $\mathcal{R}_q$ if

$$f(x) \star F_q(x) \equiv 1 \pmod{q,\ x^N - 1}.$$

When N and q are both prime, the inverse exists if and only if $\gcd(f(x),\ x^N - 1) = 1$ in $(\mathbb{Z}/q\mathbb{Z})[x]$, and can be computed using the **extended Euclidean algorithm** for polynomials. If f has no inverse, a new f is chosen.

8 Cryptosystem

8.1 Public Parameters

The system is set up with public parameters (N, p, q, d) satisfying:

- N is prime.
- $\gcd(p, q) = 1$ and $\gcd(N, q) = 1$.
- $q > (6d + 1)p$ (required for correct decryption; see Theorem 8.1).

8.2 Key Generation

Alice generates her keys as follows:

1. Choose random private polynomials $f(x) \in T(d + 1, d)$ and $g(x) \in T(d, d)$.
2. Compute inverses of f in both quotient rings:

$$F_q(x) = f(x)^{-1} \text{ in } \mathcal{R}_q, \qquad F_p(x) = f(x)^{-1} \text{ in } \mathcal{R}_p.$$

3. Compute the **public key**:

$$h(x) \equiv F_q(x) \star g(x) \pmod{q}.$$

Private key: $(f,\ F_p)$. **Public key:** h.

8.3 Encryption

Bob represents his plaintext as $m(x) \in \mathcal{R}$ with coefficients in $\left[-\frac{p}{2}, \frac{p}{2}\right]$.

1. Choose a fresh random ephemeral polynomial $r(x) \in T(d, d)$.
2. Compute the **ciphertext**:

$$e(x) \equiv p \cdot r(x) \star h(x) + m(x) \pmod{q}.$$

Because $r(x)$ is drawn fresh for each encryption, NTRU is a *probabilistic* scheme: encrypting the same message twice yields completely different ciphertexts.

8.4 Decryption

Alice recovers $m(x)$ from $e(x)$ in three steps:

1. Compute $a(x) \equiv f(x) \star e(x) \pmod{q}$.
2. **Center-lift:** adjust each coefficient of $a(x)$ to its representative in $\big(-q/2,\, q/2\big]$.
3. Recover plaintext: $m(x) \equiv F_p(x) \star a(x) \pmod{p}$.

8.5 Correctness

Theorem 8.1 (Decryption correctness)**.** *If $q > (6d+1)p$, the decryption procedure recovers $m(x)$ exactly.*

Proof. Expanding $a(x)$ and substituting $h(x) = F_q(x) \star g(x)$:

$$\begin{aligned} a(x) &\equiv f(x) \star e(x) \pmod{q} \\ &\equiv f(x) \star \big(p \cdot r(x) \star h(x) + m(x)\big) \pmod{q} \\ &\equiv p \cdot f(x) \star F_q(x) \star g(x) \star r(x) + f(x) \star m(x) \pmod{q} \\ &\equiv p \cdot g(x) \star r(x) + f(x) \star m(x) \pmod{q}, \end{aligned}$$

where the last step uses $f \star F_q \equiv 1 \pmod{q}$.

The coefficients of $A(x) := p \cdot g \star r + f \star m$ in $\mathcal{R}$ (before reduction mod q) are now bounded. Since $g \in T(d,d)$ and $r \in T(d,d)$, each coefficient of $g \star r$ is at most $2d$ in absolute value. Since $f \in T(d+1,d)$ and the coefficients of m are in $\big[-\frac{p}{2}, \frac{p}{2}\big]$, each coefficient of $f \star m$ is at most $(2d+1)\frac{p}{2}$ in absolute value. Therefore:

$$|A_k| \le p \cdot 2d + (2d+1)\frac{p}{2} < (6d+1)\frac{p}{2} < \frac{q}{2}.$$

Since every coefficient of $A(x)$ already lies in $(-q/2, q/2]$, the center-lift recovers $A(x)$ *exactly* in $\mathcal{R}$ with no modular wrap-around:

$$a(x) = p \cdot g(x) \star r(x) + f(x) \star m(x).$$

Reducing modulo p kills the blinding term ($p \cdot g \star r \equiv 0 \pmod{p}$):

$$F_p \star a \equiv F_p \star f \star m \equiv m \pmod{p}. \qquad \square$$

9 Worked Example

The full protocol is illustrated with the toy parameters $(N, p, q, d) = (7, 3, 41, 2)$.

9.1 Key Generation

Alice chooses:

$$f(x) = -1 + x^2 + x^3 - x^4 + x^6 \in T(3,2), \quad g(x) = -x - x^2 + x^4 + x^6 \in T(2,2).$$

Using the extended Euclidean algorithm she computes:

$$F_p(x) = f(x)^{-1} \pmod{3,\ x^7 - 1} = 1 + x + x^2 + x^3 + 2x^5 + x^6,$$
$$F_q(x) = f(x)^{-1} \pmod{41,\ x^7 - 1} = 37 + 2x + 40x^2 + 21x^3 + 31x^4 + 26x^5 + 8x^6.$$

The public key is:

$$h(x) = F_q(x) \star g(x) \pmod{41} = 30 + 26x + 8x^2 + 38x^3 + 2x^4 + 40x^5 + 20x^6.$$

9.2 Encryption

Bob's plaintext and ephemeral key:

$$m(x) = -x^5 + x^3 + x^2 - x + 1, \qquad r(x) = x^6 - x^5 + x - 1.$$

He computes the ciphertext $e(x) \equiv 3 \cdot r(x) \star h(x) + m(x) \pmod{41}$:

$$e(x) = 25 + 3x + 40x^2 + 2x^3 + 4x^4 + 19x^5 + 31x^6.$$

9.3 Decryption

Step 1. Multiply by $f(x)$ mod 41:

$$f(x) \star e(x) \equiv 40 + x + 40x^2 + 40x^3 + 33x^4 + 10x^5 + x^6 \pmod{41}.$$

Step 2. Center-lift to $(-20.5,\ 20.5]$ (e.g. $40 \mapsto -1$, $33 \mapsto -8$):

$$a(x) = -1 + x - x^2 - x^3 - 8x^4 + 10x^5 + x^6.$$

Step 3. Multiply by $F_p(x)$ mod 3:

$$F_p(x) \star a(x) \pmod{3} = -x^5 + x^3 + x^2 - x + 1 = m(x). \quad \checkmark$$

10 Security Analysis

10.1 Key Recovery Problem

Definition 10.1 (NTRU Key Recovery Problem)**.** Given the public key $h(x) \in \mathcal{R}_q$, find ternary polynomials $f(x)$ and $g(x)$ such that $f(x) \star h(x) \equiv g(x) \pmod{q}$.

Since $h = F_q \star g$, knowing f immediately yields $g = f \star h \pmod{q}$ (and vice versa). The difficulty of this problem is equivalent to finding a short vector in the **NTRU lattice**.

10.2 Brute Force and Meet-in-the-Middle

An attacker could exhaustively search over all $f \in T(d_1, d_2)$. The search space has size:

$$\binom{N}{d_1}\binom{N-1}{d_2} = \frac{N!}{d_1!\,d_2!\,(N-d_1-d_2)!}.$$

Note that any of the N cyclic rotations of a valid f also decrypts correctly, so the effective search space is $\#T(d_1, d_2)/N$.

Example 10.1 ($(N, p, q, d) = (251, 3, 257, 83)$)**.** The search space for $f \in T(84, 83)$ is $\approx 2^{389}$ polynomials; accounting for $N = 251$ rotations gives roughly $2^{381.6}$ effective trials. A meet-in-the-middle (birthday) attack takes the square root, requiring $\approx 2^{190.8}$ operations—still completely infeasible.

11 Conclusion

NTRU is a mature, well-studied post-quantum cryptosystem. Its algebraic foundation in truncated polynomial rings provides a clean mathematical structure, and its reliance on lattice hardness gives resilience against both classical and quantum adversaries. Its combination of small key sizes, fast operations, and strong security evidence makes it one of the most practical post-quantum proposals available today.

This section forms part of a group report on Post-Quantum Cryptography. The initial content was developed from a presentation by Meghraj Goswami.

References

[1] P.W. Shor. *Algorithms for Quantum Computation: Discrete Logarithms and Factoring.* In: *Proceedings of the 35th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 1994, pp. 124–134. https://doi.org/10.1109/SFCS.1994.365700

[2] J. Hoffstein, J. Pipher, and J.H. Silverman. *NTRU A Ring-Based Public Key Cryptosystem.* In: *Algorithmic Number Theory (ANTS-III)*, LNCS 1423, Springer, 1998, pp. 267–288. https://doi.org/10.1007/BFb0054868

[3] *NIST Post-Quantum Standardization Effort.* https://csrc.nist.gov/projects/post-quantum-cryptography